AI Edge Inference using executorch

Quantization| Executorch | HuggingFace | Android

Model Compressor

LLM-driven layer analysis selects optimal INT8/FP16 or pruning strategy via torch.export + ExecuTorch. Emits a verified .pte file.

HuggingFace Uploader

File-watches Agent 01's output. Uploads to your HF repo, polls until SHA256 checksum passes, then fires a structured handoff event.

Android Studio Builder

Scaffolds an Android project from a PyTorch Mobile template, injects the HF model URL, runs Gradle, and auto-fixes build errors — up t

WiFi Bridge

Uses mDNS/zeroconf to discover your device, pairs via adb pair through a rich TUI. Falls back to USB automatically when wireless isn't

Deploy Orchestrator

Runs gradlew installDebug on your wireless device, tails logcat for confirmed launch, and presents the final "app is live" prompt.

Export Phi-3 → ExecuTorch (.pte)

- Script: createtest2.py (export_phi3 flow)
- Goal: produce a quantized ExecuTorch `.pte` bundle optimized for Android (XNNPACK / ARM NEON)
- Target: memory-constrained devices (example: Phi-3 Mini → ~900 MB runtime)

Pipeline (what the script does)

1. Ensure weights & custom arch files present (downloads from HF if needed) → copies tokenizer.json to output
2. Patch config.json (rope_scaling, use_cache=False) to make export-safe
3. Load HF model + tokenizer (trust_remote_code / local files) and apply small runtime patches
4. Patch RotaryEmbedding (RoPE) to remove data-dependent branches for torch.export
5. Quantize model (8da4w via torchao)
6. Capture computation graph via torch.export (dynamic seq dim)
7. Lower to Edge IR & delegate to XNNPACK via ExecuTorch
8. Serialize to `.pte` FlatBuffer and run a quick sanity inference via executorch.runtime

Quantization (8da4w)

- Method: Int8 dynamic activations + Int4 weights (script uses Int8DynActInt4WeightQuantizer(groupsize=32))
- Implemented by: torchao.quantization (applies to HF model before export)
- Benefits:
  - Large reduction in model size (weights → INT4) and runtime memory (activations dynamic INT8)
  - Makes on-device inference feasible on <4GB RAM phones
  - Faster CPU inference with XNNPACK on ARM NEON
- Tradeoffs:
  - Small loss in numeric precision / potential quality drop
  - Must verify behavior (sanity checks / end-to-end tests)

ExecuTorch & XNNPACK (why and how)

- ExecuTorch role:
  - Accepts the exported Torch program → transforms to Edge IR → partitions and lowers
  - Produces a compact, deployable FlatBuffer `.pte` runtime artifact
  - Provides a small runtime for Android to load and run the model
- XNNPACK:
  - Partitioned backend (via XnnpackPartitioner) that implements CPU-optimized kernels (ARM NEON)
  - Improves throughput and reduces memory overhead on mobile CPUs
- Final output: `*.pte` + tokenizer.json → copied into Android app assets

Slide 5 — Demo / Run & Notes

- Example run:

```
HF_HUB_DISABLE_XET=1 python createtest2.py --model phi3 --max_seq_len 512
```

- Main outputs (in android_assets by default):
  - `phi3_mini_8da4w.pte` (serialized ExecuTorch artifact)
  - tokenizer.json
  - `export_report.txt`
- Tips:
  - Recommended stack (script tested): torch 2.11, torchao 0.17, executorch 1.2
  - Use smaller --max_seq_len (e.g., 128) for faster iteration
  - Copy `.pte` & tokenizer.json into your Android app assets for deployment

```
# Paste your token: Settings → Access Tokens → New token (Write permission)

huggingface-cli repo create phi3-mobile-model --type model
```

2. Uplocad the .pte file:

```
huggingface-cli upload akhilsalogra/phi3-mobile-model \
  ./android_assets/phi3_mini_8da4w.pte \
  model.pte
```

3. Upload tokenizer.json:

```
huggingface-cli upload akhilsalogra/phi3-mobile-model \
  ./android_assets/tokenizer.json \
  tokenizer.json
```

4. Use in MainActivity.kt

```
private val MODEL_URL =
    "https://huggingface.co/akhilsalogra/phi3-mobile-model/resolve/main/model.pte"
```

Android Studio

MainActivity is the Android UI + runtime glue for an on-device Phi-3 chatbot. It downloads a prebuilt, quantized ExecuTorch `.pte` model + tokenizer, loads the model via ExecuTorch's `LlamaModule`, streams token-by-token generations to the UI, and exposes simple conversation state and progress reporting. Key goals: make a large LLM run locally on mobile (no server), show download/progress, warm-up the runtime, and stream outputs to the chat UI

```
EdgeApp/
├── build.gradle                        ← project-level (Kotlin plugin, AGP version)
├── settings.gradle                     ← declares :app module, Maven repos
├── gradle.properties                   ← org.gradle.jvmargs, Kotlin options
│
└── app/
    ├── build.gradle                    ← app deps: executorch-android, coroutines
    │
    └── src/main/
        ├── AndroidManifest.xml         ← INTERNET permission, launcher activity,
        │                                 networkSecurityConfig reference
        ├── java/com/example/phi3edge/
        │   ├── MainActivity.kt         ← download .pte, cache, run inference
        │   └── LlmEngine.kt           ← (optional) wraps Module.load/forward
        │
        ├── res/
        │   ├── layout/
        │   │   └── activity_main.xml    ← TextView (status), EditText, Button, output
        │   ├── values/
        │   │   └── strings.xml
        │   └── xml/
        │       └── network_security_config.xml   ← allow HF download (http → https)
        │
        ├── assets/                     ← model.pte + tokenizer.json land here
        │                                 (after first-launch download via filesDir)
        └── jniLibs/arm64-v8a/
            └── libexecutorch.so        ← auto-extracted by Gradle from AAR dep
```

- Purpose
  - Single Activity that ties UI, downloads, and model runtime together for an on-device chatbot (`com.example.phi3chat`).
  - Works with a prepackaged quantized model file `phi3_mini_8da4w.pte` and tokenizer.json.
- App lifecycle & native init
  - Calls `SoLoader.init(...)` to initialize native libraries early (ExecuTorch native code).
  - Cleans up `LlamaModule` and receiver in `onDestroy()`.
- UI & UX
  - Standard chat UI: RecyclerView + `ChatAdapter`, EditText for input, send/clear buttons.
  - Controls are disabled while model loads; enabled when ready.
  - Streaming UI: adds a placeholder message and updates it token-by-token during generation.
- Downloading model & tokenizer
  - Uses Android `DownloadManager` to fetch large files (model ~2+ GB).
  - Registers a BroadcastReceiver for ACTION_DOWNLOAD_COMPLETE.
  - Polls download progress with a coroutine (updates a ProgressBar and status text).
  - Checks file sizes on completion before attempting load (simple integrity heuristic).
- File storage
  - Saves model/tokenizer with `getExternalFilesDir(Environment.DIRECTORY_DOWNLOADS)` (or `filesDir` as fallback).
  - Prompts and guidance logged in chat when download starts/completes.

- Loading the model (ExecuTorch integration)
  - Instantiates `LlamaModule(modelPath, tokenizerPath, TEMPERATURE)` on a background coroutine (IO dispatcher).
  - Performs a short warm-up generate call to initialize weights/kernels (and produce runtime caches).
  - Sets `isModelLoaded = true` and enables input when ready.
- Generation & streaming
  - Builds instruction-style prompt including system token + conversation history.
  - Calls `llamaModule.generate(prompt, MAX_NEW_TOKENS, LlamaCallback)` on IO dispatcher.
  - `LlamaCallback.onResult` appends tokens to a buffer and updates UI on the main thread for streaming effect.
  - `onStats` reports throughput (tok/s) to UI.
  - Conversation history is appended using the model's expected token delimiters and preserved across turns.
- Concurrency & coroutines
  - All heavy operations run in `lifecycleScope.launch(Dispatchers.IO)` to avoid UI blocking.
  - UI updates happen via `withContext(Dispatchers.Main)` or `lifecycleScope.launch` from callbacks.
  - Polling uses `delay()` in coroutine to periodically refresh progress.
- Error handling & UX fallbacks
  - Try/catch around model init and generate; updates UI with friendly error messages.
  - If downloads are missing or incomplete, shows a download message rather than crashing.
- Security & config notes
  - `HF_TOKEN` placeholder supports private Hugging Face downloads (Authorization header added to requests if set).

Pair new devices over Wi-Fi

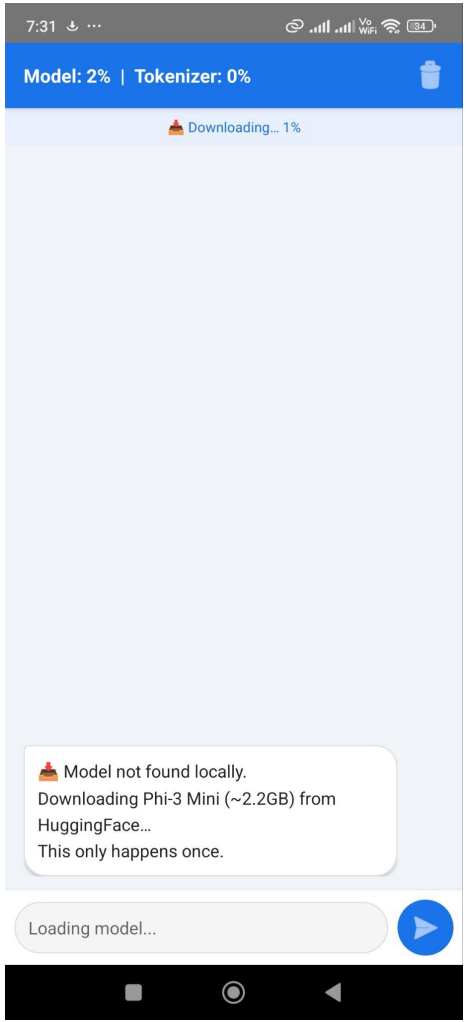
Pair devices to enable wireless debugging. Other devices can be paired using a pairing code. Learn more.

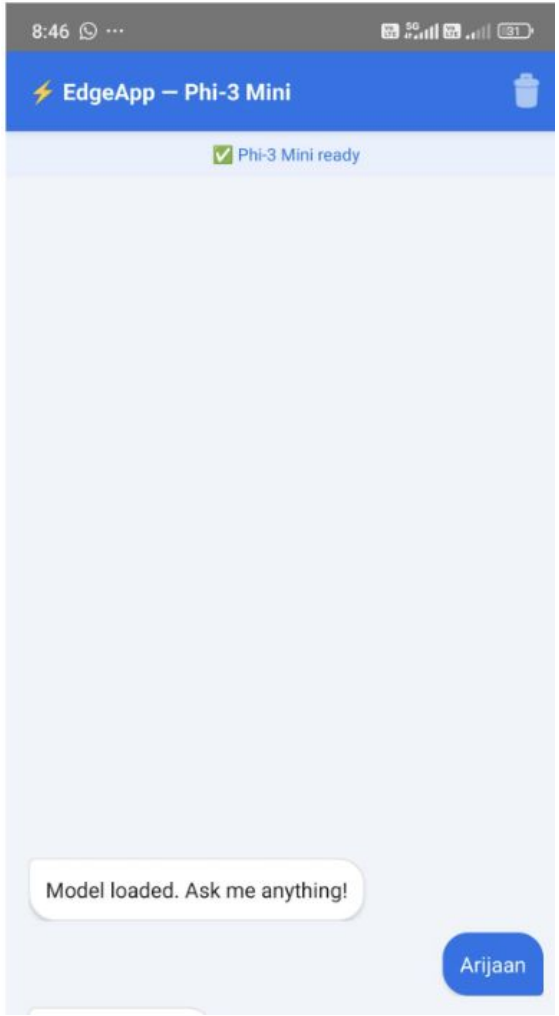
Pair using QR code | Pair using pairing code

To pair an **Android 11+** device
scan the QR code from your device

QR scanner available at:
Developer options > Wireless debugging > Pair using QR code

BASE_URL/$MODEL_FILENA
BASE_URL/$TOKENIZER_FI
rView
t
tton
tton
le.phi3chat
t × Build Analyzer ×
hed At 08/04/26, 8:50 pm wi
ugKotlin 1 warning
t app/src/main/java/com/exa
compatible with Gradle
ion warnings and deter


7:31
Model: 2% | Tokenizer: 0%
Downloading... 1%
Model not found locally.
Downloading Phi-3 Mini (~2.2GB) from HuggingFace...
This only happens once.
Loading model...


8:46
EdgeApp — Phi-3 Mini
Phi-3 Mini ready
Model loaded. Ask me anything!
Arijaan